

Search for the Egg

Development-grade design, systems, economy, architecture, monetization, game-feel, implementation phases, assumptions, and verification standards.

PRODUCTION DECISION

Proceed now using centralized ASSUMED behavior for evidence gaps. No additional footage is required to begin.

Document controls

Field	Value
Target	Search for the Egg
Reference	Search For The Needle by Garage Games
Evidence	User recording plus public current/historical research
Evidence date	12 September 2026
Implementation	Six controlled Claude Code phases
Authority	MASTER_SPEC.md and SYSTEM_CONFIG.json

Premium bar

Every phase must include purposeful animation, ambient movement, layered audio, responsive UI, satisfying feedback, strong lighting, polished VFX, environmental activity, cleanup, and mobile-aware performance. Phase 6 completes and measures polish; it does not introduce it for the first time.

Contents

Document controls	1
Premium bar	1
Contents	2
Search for the Egg	5
Final Game Design Document	5
Executive decision	5
Product vision	5
Build thesis	5
Evidence basis	5
Evidence labels	6
Six-phase roadmap	6
Human and machine-readable deliverables	6
Observed-build visual evidence	7
Search for the Egg - Master Implementation Specification	8
1. Document authority	8
2. Product definition	8
3. Non-negotiable principles	8
4. Experience structure	9
5. Player journey	10
6. Core round design	10
7. Tutorial and objective system	12
8. Tools	12
9. Progression and difficulty	13
10. Classes	13

11. Lobby systems	14
12. Chapter 2	14
13. Networking contract	15
14. Service architecture	15
15. Premium game-feel specification	17
16. UI and platform requirements	18
17. Monetization design	18
18. Analytics and balancing	19
19. Security model	19
20. Performance architecture	19
21. Six-phase implementation contract	20
22. Repository and change discipline	20
23. Definition of done	21
Economy reference tables	22
Bag progression	22
Tool acquisition	22
Completion rewards	22
Daily rewards	22
Class distribution	23
Robux products	23
Technical implementation appendices	24
Persistence and transactions	24
State-machine invariants	24
Verification model	24
Search for the Egg - Assumptions Register	25
Rules for using assumptions	25

Core round assumptions 25

Egg discovery assumptions 25

Economy and progression assumptions 26

Class and RNG assumptions 26

Daily, codes, and event assumptions 26

Chapter 2 assumptions 27

Persistence assumptions 27

Multiplayer assumptions 27

Mobile and accessibility assumptions 27

Premium presentation assumptions 28

Explicitly disabled until separately designed 28

Research sources 29

Clean-room restriction 29

Search for the Egg

Final Game Design Document

Document version: 1.0 Research and evidence date: 12 September 2026 Production status: Approved to build in six controlled phases with centralized ASSUMED values

Executive decision

Search for the Egg will enter production now. The supplied five-minute recording and public research are the source of truth for observable lobby, onboarding, collection, selling, tool, upgrade, class, event, and monetization behavior. The recording does not contain the post-victory journey, Hard Mode, Chapter 2, persistence, multiplayer, or mobile behavior. Those systems are deliberately designed here using strong Roblox conventions and are marked ASSUMED; their values and policies are centralized in `SYSTEM_CONFIG.json`.

This is not an evidence-gap report. It is a production specification.

Product vision

Players enter a living, oversized nest environment and use increasingly powerful methods to clear feathers while searching for one hidden Egg. The tactile loop is immediately understandable, but the hidden target creates suspense: every removed patch might expose the Egg. Within a round, the player accelerates through selling and upgrades. Across rounds, wins, best times, classes, permanent perks, chapters, difficulty, cosmetics, events, and daily rewards create return goals.

The product must feel premium and authored. No primary area may resemble an empty simulator template. Animation, ambient movement, layered audio, responsive UI, lighting, VFX, camera response, haptics, and environmental activity are treated as system requirements from the first playable phase. Effects are pooled, distance-culled, quality-scaled, and compatible with mobile performance.

Build thesis

The game's commercial and retention potential rests on five connected beats:

1. Immediate tactile collection with visible world deformation.
2. Frequent short upgrades that noticeably change efficiency.
3. A concealed Egg that turns ordinary collecting into suspense.
4. A short, high-quality discovery and victory payoff.
5. Persistent classes, wins, unlocks, rewards, and events that make another run meaningful.

Evidence basis

Primary direct evidence is the user-supplied 5:25.667 H.264 recording at 1280x720 and 60 FPS. It captures a fresh profile, approximately one minute of lobby/meta surfaces, an 18-20 second chapter transition/cutscene, and roughly four minutes of active Farmhouse play. It directly supports the starting 25 capacity, baseline sale examples, early upgrade prices, tool packaging, class odds, Daily rewards, event chest odds, party capacity, and visible unlock gates.

Public sources establish the official experience identity, universe/place structure, badge signals, current analytics snapshot, class-effect corroboration, event codes/currency guidance, Chapter 2 route hypothesis, and historical pass descriptions. Version conflicts are resolved in favor of the uploaded live recording for visible values.

Evidence labels

- **VERIFIED**: directly visible in the recording or officially documented.
- **CORROBORATED**: supported by multiple reliable public sources or video plus a reliable guide.
- **ASSUMED**: selected production behavior where exact current-build evidence is unavailable.

ASSUMED never means hidden. It means configurable, testable, and easy to revise.

Six-phase roadmap

1. Architecture, networking, configuration, player data, sessions, and tests.
2. Complete playable lobby-to-victory Chapter 1 vertical slice.
3. Progression, bags, Hand, all tools, upgrades, economy, and difficulty math.
4. Complete lobby/meta systems, parties, classes, rewards, inventory, stats, and selection.
5. Hard Mode, Moonlit Cellar/Chapter 2, monetization, permanent tools, and products.
6. Mobile/controller, multiplayer edge cases, security, optimization, complete polish, balance, and release QA.

Every phase must finish with automated tests, actual Roblox Studio verification, concrete evidence, updated documentation, and a committed checkpoint. Claude Code must stop for approval before the next phase.

Human and machine-readable deliverables

This PDF explains product intent and design. Implementation must also follow:

- **MASTER_SPEC.md**
- **SYSTEM_CONFIG.json**
- **ECONOMY_TABLES.csv**
- **DATASTORE_SCHEMA.md**
- **STATE_MACHINES.md**
- **TEST_PLAN.md**
- **ASSUMPTIONS.md**
- Six standalone phase prompts
- **START_HERE.md**

If this PDF and the machine-readable files conflict, the authority order in **MASTER_SPEC.md** applies.

Observed-build visual evidence

These frames are from the supplied recording and establish the visual/UI source boundary. They are evidence, not assets for reuse.



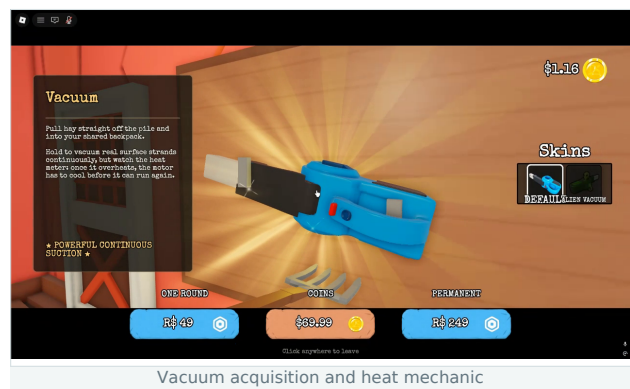
Daily rewards and live update countdown



Class odds and roll surface



Compact Farmhouse collection space



Vacuum acquisition and heat mechanic

Observed facts include the 25-feather starting bag, class weights totaling 100%, Daily reward amounts, four tool families, party capacity up to four, a count-up round timer, Chapter/Hard one-win locks, and visible purchase packaging. Completion and later systems are explicitly assumed in the build pack.

Search for the Egg - Master Implementation Specification

Version: 1.0 Evidence snapshot: 12 September 2026 Status: Approved for phased implementation with centralized ASSUMED values

1. Document authority

This file is the implementation source of truth for Claude Code. Resolve conflicts in this order:

1. The user's latest explicit instruction.
2. MASTER_SPEC.md for behavior and architecture.
3. SYSTEM_CONFIG.json for values, IDs, feature flags, prices, rewards, and thresholds.
4. STATE_MACHINES.md for transitions and invariants.
5. DATASTORE_SCHEMA.md for persistence and transaction rules.
6. TEST_PLAN.md for acceptance evidence.
7. ASSUMPTIONS.md for provenance and changeability.
8. ECONOMY_TABLES.csv as a balancing view of the JSON; if values disagree, stop and report the conflict.
9. FINAL_GDD.pdf for human-readable intent and presentation.

Claude Code must read the entire pack before modifying the repository. It must inspect the existing codebase and preserve healthy systems. No phase authorizes a full rewrite.

2. Product definition

Search for the Egg is a cozy first-person Roblox collection and hidden-object game. Players enter compact, richly animated environments containing enormous feather piles or nests. They physically clear feathers, fill a personal bag, sell feathers for round cash, buy stronger collection methods, and search for the hidden Egg. The first valid Egg discovery ends the party's round, awards progression, and returns the party to the lobby.

The target pursues close functional parity with the observable structure of Search For The Needle while using entirely original branding, world construction, art, UI, models, animation, audio, text, and code.

Core fantasy

"I am digging through an impossibly large living nest, getting visibly stronger every minute, and any removed feather could reveal the Egg."

Player promise

- Immediate tactile feedback from the first click/tap.
- A legible short-term target: fill, sell, upgrade, search deeper.
- Constant visual proof that the party is changing the pile.
- Suspense from a concealed server-selected Egg position.
- A concise, premium discovery and victory payoff.
- Persistent reasons to return: wins, best times, classes, daily rewards, cosmetics, Hard mode, and Chapter 2.

3. Non-negotiable principles

Server authority

The server decides pile cells removed, feathers awarded, bag limits, sale value, cash, upgrade eligibility, tool ownership, cooldowns, RNG, Egg position, winner, time, rewards, unlocks, receipt grants, and saved data. Client-side prediction is cosmetic.

Configuration first

All tunable values must originate from a typed configuration module generated or manually mirrored from `SYSTEM_CONFIG.json`. Services may cache resolved values; they may not restate literals. Product IDs are deployment-specific and must be provided through a separate secrets/deployment mapping, never guessed.

Assumption visibility

Unverified behavior is allowed and required to unblock production. It must be marked `ASSUMED` in docs/config and isolated behind policies or values that can be changed without architectural surgery.

Premium from Phase 1

Polish is not a Phase 6 paint job. Each vertical implementation includes responsive input, animation timing, audio cue, VFX cue, camera/UI response, ambient world behavior, quality scaling, and cleanup. Phase 6 unifies, measures, replaces placeholders, and optimizes these layers.

Original expression

Do not copy reference assets, layouts pixel-for-pixel, code, map geometry, character names, UI text, icons, audio, or models. Functional behaviors may be reconstructed; expression must be original.

4. Experience structure

Places

Preferred production structure:

1. Lobby place: social/meta hub, party formation, shops, classes, rewards, inventory, stats, chapter selection.
2. Chapter 1 place: Sunlit Henhouse.
3. Chapter 2 place: Moonlit Cellar.

During early development these may coexist in one test place behind adapters. The architecture must not assume same-place travel. Teleport data contains opaque reservation/session identifiers only.

Server sizes

- Lobby: maximum 20 players.
- Round party: 1-4 players.
- One authoritative round per reserved gameplay server unless profiling proves safe multi-round hosting.

Chapter 1 - Sunlit Henhouse

A warm barn/henhouse centered on a huge sculpted nest. A collection/selling character or station sits across a short movement lane. Upgrade stations form an intuitive loop around the nest. Windows, fans, hanging cloth, pecking birds, floating dust, drifting feathers, moving belts, and distant farm activity make the space alive without obscuring interactions.

Chapter objective: clear the nest, expose the hidden Egg, and claim it first for the party.

Chapter 2 - Moonlit Cellar

An original subterranean hatchery/cellar. The party searches feather and nesting debris for a hidden key, activates three mechanisms, solves three compact shared puzzles, releases a trapped hatchling, and exits with the Egg. The public reference structure is inspiration for functional pacing only; all layout, story, puzzle presentation, symbols, models, and copy must be original.

5. Player journey

First session

1. Load profile and entitlements before interactive spawn.
2. Spawn facing the primary chapter portal and visible Daily reward cue.
3. Display one concise objective: create/join a Chapter 1 Normal party.
4. Party start -> reservation -> teleport/load.
5. One skippable 18-second story scene establishes the lost Egg.
6. Tutorial asks the player to collect 25 feathers.
7. Bag fills, sell station highlights, player sells.
8. Upgrade board highlights; player buys a starter upgrade when affordable.
9. Tutorial collapses into the standard objective: "Search the nest for the Egg."
10. The player cycles collection, selling, and upgrades.
11. Covering cells reveal the Egg; a positional chime and glint create the discovery moment.
12. Claim -> winner announcement -> committed rewards -> results -> lobby.
13. First win unlocks Chapter 1 Hard and Chapter 2 Normal through configuration policy.

Short-session loop

Lobby -> party -> chapter -> collect -> sell -> upgrade -> reveal -> win -> rewards -> replay/lobby.

Long-term loop

Earn wins/Gems/tokens -> roll and equip classes -> improve permanent perks -> unlock chapters/difficulties -> improve best time -> collect original tool cosmetics/event rewards -> return for daily/event cadence.

6. Core round design

Timer

Timer counts upward from the authoritative **Searching** start timestamp. Display hundredths only where it improves time-record excitement; server stores integer milliseconds. Cutscene/countdown time is excluded.

Pile model

Do not construct the pile from thousands of networked physical feathers. Use two layers:

1. Server logical grid/volume: roughly 256 cells, each with bounds, remaining density/health, region, and optional spawn-node coverage relationship.
2. Client visual layer: pooled meshes/strands and surface chunks driven by cell snapshots/deltas.

The server accepts an action origin/direction/tool ID, identifies valid nearby cells, removes an allowed amount, and returns deltas plus awarded counts. Clients animate strands, chunks, compression, particles, and small debris.

Collision must remain simple and stable.

Requirements:

- Removed volume is visible and persistent to the party.
- New clients receive a snapshot then deltas.
- Simultaneous hits cannot award the same cell content twice.
- Cell removal and award calculation are atomic within the round service.
- Mobile visual density is lower without changing authoritative yield.
- Egg cover is defined by logical cells, not unreliable touch events.

Bag

Starting capacity is 25. Normal collection clamps to free space. Tool-generated overflow becomes short-lived server-owned collectible bundles if enabled, otherwise only the fitting amount is awarded. Never display impossible values such as 139/25 as normal bag state.

Bag capacity is calculated in this order:

```
TEXT SPECIFICATION
roundBagLevelCapacity
* permanentBagPerkMultiplier
* classBagMultiplier
or Infinite Bag entitlement override
```

Round displayed capacity is floored to an integer. Infinite capacity uses an explicit state, not an enormous sentinel exposed to UI.

Selling

Interact with the selling station to exchange all personal carried contents. The server computes integer cents and returns a transaction result containing counts, modifiers, and final cents. UI animates bag decrement and cash increment from that result.

Egg spawn and reveal

At round bootstrap, the server selects a validated chapter spawn node using the configured weighted policy and recent-node protection. Nodes define transform, cover cells, reveal camera anchor, and allowed depths.

The Egg remains non-interactive and visually concealed until its cover condition is met. On reveal, join is closed and the party sees a synchronized but quality-scaled cue. The first valid claim wins atomically. A party receives shared base rewards; the finder receives the configured personal bonus.

Victory

Use the standard assumed flow exactly:

Egg discovered -> winner announcement -> round rewards -> results screen -> return to lobby -> progression saved

Reward persistence is committed before teleport. If saving is delayed, keep the results scene alive with a polished "Securing rewards..." state. Never show a paid or earned reward as final based only on a client animation.

Results contain:

- Winner/finder.

- Chapter and difficulty.
- Authoritative completion time.
- New personal best marker.
- Individual contribution.
- Base reward and finder bonus separated.
- Newly unlocked content.
- Replay and Lobby actions.

7. Tutorial and objective system

Objectives are server-approved state IDs with localized client copy. They are not raw server strings.

First-run steps:

1. `enter_party`
2. `collect_25`
3. `sell_first_bag`
4. `buy_first_upgrade`
5. `search_for_egg`
6. `claim_or_complete_round`

Tutorial highlights one target at a time, never covers the reticle/action area, respects safe areas, and survives character respawn. It can be skipped after the story setup; completion state persists by tutorial version.

8. Tools

Target tools use original names and presentation while preserving distinct roles.

Hand

Free baseline. Responsive short reach, exact server cooldown, local hand animation, contact puff, feather tug, and subtle haptic. Paths: Grasp, Speed, Hold.

Nest Rake

Deep discrete extraction. Slower cadence, wider/deeper server query, larger sculpted dent, weighty anticipation and recovery. Paths: Sweep, Cooldown, Hold.

Confetti Charge

Burst area clearing. Light, throw, fuse, explosion, surface displacement, optional rainbow conversion. Server owns charge/fuse/affected cells. Paths: Lucky Blast, Speed, Power. Visual theme is playful paper/confetti pressure rather than copied explosives.

Feather Vac

Continuous surface collection with runtime/heat and forced cooling. Server integrates allowed suction over time; client smooths beam/strand motion and meter. Paths: Power, Cooling, Runtime.

Scout Chick

Autonomous helper that travels to the nest, fills its own capacity, returns to the seller, and deposits value for its owner. Server chooses cells and grants output. Client renders smoothed motion and personality animation. Paths: Speed, Grasp, Capacity.

Acquisition and reset

- Soft-currency and one-round Robux acquisition lasts for the current round only.
- Permanent entitlement grants the tool automatically at round start and currently assumes max tool upgrades.
- Every rule is centralized in configuration.
- Product ownership is server-checked.

9. Progression and difficulty

Round progression

Cash is round-scoped. Players buy Hand, Bag, tool access, and tool upgrades. This creates acceleration within one run while preventing early permanent power from erasing the search loop.

Persistent progression

Gems/tokens buy class rolls, permanent perks, event chests, and eligible cosmetics. Wins and best times unlock difficulties/chapters and support leaderboards.

Difficulty

Normal is default. Hard is unlocked through the policy object in configuration, never through scattered `wins >= 1` checks. Hard currently assumes deeper Egg placement, more pile density, slightly lower tool yield, improved sale value, and larger completion reward. All modifiers compose through one resolved round-modifier object.

10. Classes

Implement the eight configured target classes with the verified probability weights:

- Newcomer 40%
- Nest Carrier 25%
- Egg Trader 14%
- Rake Expert 9%
- Blast Artist 7%
- Gem Seeker 3.9%
- Automation Expert 1%
- Master Forager 0.1%

Server selects the result before animation. Client wheel/card animation lands on the committed result, then the server projection updates and reward modal appears. The result must not be announced before the animation lands.

Class modifiers are composed by a pure `ModifierResolver` with named sources. No service manually checks class names. The resolver returns bag, sale, tool, proc, and entitlement modifiers and prevents double application.

The assumed 2x Luck transformation is isolated in one policy function and unit-tested for non-negative weights and exact normalization.

11. Lobby systems

Layout

The lobby is compact and readable from spawn. Primary portal/party interaction is straight ahead. Daily reward and classes are visible from the main path. Shops and event chest flank the route without forming a dead mall. Stats/leaderboards and social reward occupy secondary sightlines.

Every station has world animation plus UI identity:

- Party portal: animated destination diorama and current party silhouette.
- Daily rewards: opening calendar/nest display and ready pulse.
- Classes: rotating class tokens/cards, visible odds, reveal stage.
- Permanent perks: mechanical upgrade bench with before/after previews.
- Event chest: contained UFO/moonlight effect with strict particle budget.
- Inventory: physical display hooks and a responsive modal.
- Codes/Stats: secondary stations, not dominant spawn clutter.

Parties

Create/join, leader, 1-4 players, chapter, difficulty, unlock validation, countdown, reservation, and teleport fallback. Party actions are server-serialized. Invite features may integrate Roblox-native invite APIs behind an adapter.

Daily rewards

Eight-day visible cycle with verified Gem amounts. Timing, grace, loop, and miss policy are assumed/configured. Server time and one claim transaction are mandatory.

Codes

Codes are normalized uppercase, trimmed, rate-limited, server-configured, optionally time-bounded, and idempotent per account. Never expose unpublished codes to the client config.

Inventory

Categories: Rake, Charge, Vac, Chick, and future cosmetics. Client requests equip by owned item ID; server validates ownership and returns the new projection. Cosmetics never change power unless a separately named gameplay entitlement says so.

Stats and leaderboards

Personal stats: playtime, equipped class, best times, total wins, feathers collected, Eggs found. Leaderboards use bounded update cadence and ordered stores; they do not write on every feather.

12. Chapter 2

Phase 5 implements a complete assumed/corroborated Chapter 2 journey:

1. Search debris for a hidden key.
2. Activate three spatially separated mechanisms.
3. Solve a visible-object count puzzle.
4. Assemble a four-digit code from environmental notes.

5. Rotate a 4x4 tile image into the correct orientation.
6. Release a trapped hatchling and disable the barrier.
7. Exit with the Egg and trigger the same transaction-safe victory pipeline.

Puzzle solutions are server-generated from validated sets, shared by the party, recoverable after rejoin, and impossible to permanently softlock through wrong input. UI must not rely on color alone; shapes/symbols accompany colors.

13. Networking contract

Create one typed remote registry with schemas, rates, permissions, and direction. Suggested logical messages:

Client requests:

- `PartyCreate`, `PartyJoin`, `PartyLeave`, `PartySetDestination`, `PartyStart`
- `TutorialAction`
- `CollectAction`, `ToolEquip`, `ToolActionBegin`, `ToolActionEnd`
- `SellRequest`, `UpgradePurchase`
- `EggClaim`
- `ClassRoll`, `ClassEquip`
- `DailyClaim`, `CodeRedeem`, `GroupRewardClaim`
- `InventoryEquip`
- `ResultsAction`
- `SettingsUpdate`
- `PurchasePromptRequest`

Server events/projections:

- `ProfileProjection`
- `PartyProjection`
- `RoundSnapshot`, `RoundDelta`, `ObjectiveChanged`
- `ActionResult`, `SaleResult`, `UpgradeResult`
- `EggRevealed`, `VictoryStarted`, `RewardsCommitted`, `ResultsProjection`
- `Toast`, `ModalDirective`
- `EntitlementsProjection`

Every request schema rejects unexpected types/size, unknown IDs, stale tokens, impossible states, distance violations, and rate violations. Never create a generic remote that accepts arbitrary method names and payloads from the client.

14. Service architecture

Adapt names to the healthy existing repository rather than duplicating equivalent modules.

Shared/domain

- `Config`: typed immutable configuration and deployment product mapping.
- `Types`: domain types only.

- **NetSchema**: remote contracts and sanitizers.
- **StateReducers**: pure lobby/party/round/UI reducers.
- **EconomyMath**: integer-cents calculations.
- **ModifierResolver**: class/perk/difficulty composition.
- **WeightedRandom**: injected RNG.
- **Clock**: injected wall/monotonic time.

Server

- **PlayerDataService**
- **EntitlementService**
- **ReceiptService**
- **PartyService**
- **SessionService**
- **RoundService**
- **PileService**
- **CollectionService**
- **SellingService**
- **UpgradeService**
- **ToolService** with tool strategies
- **EggService**
- **VictoryService**
- **ChapterService**
- **PuzzleService**
- **ClassService**
- **RewardService**
- **InventoryService**
- **DailyService**
- **CodeService**
- **StatsService**
- **AnalyticsService**
- **SecurityService**

Client

- **BootstrapController**
- **InputController**
- **CameraController**
- **HUDController**
- **ModalController**
- **PartyController**
- **RoundController**
- **PileVisualController**

- ToolVisualController
- ObjectiveController
- VictoryController
- LobbyController
- AudioDirector
- VFXDirector
- HapticsController
- QualityController
- AccessibilityController

Presentation components

Use a component library with design tokens for typography, color, spacing, radius, stroke, shadow, motion, sound semantics, and input prompts. Components include buttons, tabs, currency chips, progress meters, product cards, tooltips, toast queue, modal shell, reward row, class card, party member card, and result card.

15. Premium game-feel specification

Every important action uses a feedback envelope:

1. Anticipation: windup, hover/press response, sound pickup, target highlight.
2. Contact: authoritative impact aligned to animation marker.
3. Reward: number/currency change, particles, pitch layer, haptic, visible world change.
4. Recovery: eased settle and immediate readiness cue.

Collection

- Feathers bend/tug before release, then stream toward the bag.
- Contact sound varies by tool and uses small pitch variation without becoming noisy.
- Bag meter has restrained squash and a stronger full-state pulse.
- Surface dents are spatially coherent; no random visual holes unrelated to server cells.

Selling

- Bag contents visually arc into the station.
- Cash counts with a short accelerating tick, not one sound per feather.
- Large sales use a stronger but capped burst.
- The player can move again quickly; feedback does not trap input.

Upgrades

- Preview clearly states old -> new and cost.
- Purchase produces one mechanical world response, UI snap, audio chord, and stat number transition.
- Maxed state is celebratory and visually distinct, not simply disabled grey.

Discovery/victory

- Final cover lifts in a readable spiral.

- Directional chime and glow guide all party members.
- Camera framing never causes motion sickness; reduced-motion uses a static framed cut.
- Winner and party both feel rewarded.
- Victory presentation is short enough to sustain replay.

Ambient life

- At least three visible ambient behaviors from any primary standing position.
- Mix large slow motions (fan, cloth, bird path) with small close motions (dust, feathers, props).
- Ambient actors use seeded schedules and pooling, not permanent heartbeat loops per object.
- Interactables remain the highest contrast; ambiance must not become visual noise.

Audio

- Separate buses: Music, Ambience, SFX, UI.
- State-driven music layers: Lobby, Search, Discovery tension, Victory, Results.
- Positional sounds for world actions; UI sounds remain non-positional.
- Concurrency limits and priority prevent tool spam from clipping.
- Settings apply immediately and persist.

16. UI and platform requirements

- Responsive constraints, safe areas, and UIScale policy; no hard-coded desktop-only offsets.
- Mobile primary action under comfortable right thumb; movement remains clear.
- Hold/continuous tools support touch down/up and cancellation.
- Controller actions use ContextActionService and explicit selection groups.
- Mouse, touch, and controller prompts switch without rebuilding the entire UI.
- Minimum 44 px touch targets.
- Text remains legible at small phone widths.
- No more than one blocking modal.
- Reduced motion and camera shake toggles.
- Color is never the sole puzzle/status signal.

17. Monetization design

Implement only configured products and product mapping supplied by the owner. Prices in UI come from validated config/product info, never from duplicated literals.

Principles:

- No forced purchase during tutorial.
- No purchase required to complete a chapter.
- Show exact benefit before native prompt.
- One-round versus permanent is unmistakable.
- Cancel closes cleanly and returns focus.
- Native purchase completion is not proof of grant; server ownership/receipt is.

- Paid acceleration does not create exclusive server-authoritative completion access.

Current surfaces: Gem bundles, 2x Gems, Infinite Bag, class 2x Luck, Fast Rolls, extra class slot, one-round and permanent tools, Skip Upgrade products, event chest bundles.

Developer product IDs remain `TODO_DEPLOYMENT_IDS`. Claude Code must create a validated mapping interface and refuse unknown IDs; it must not invent live IDs.

18. Analytics and balancing

Emit the configured enumerated events with build/config version, chapter, difficulty, party size, platform class, and relevant numeric state. Sample high-frequency collection telemetry. Never log free text.

Primary funnels:

- Join -> lobby interactive -> party created -> round started.
- Round started -> first collect -> bag full -> first sale -> first upgrade.
- First sale -> Egg reveal -> completion -> replay.
- Win -> Hard/Chapter 2 selection.
- Shop view -> prompt -> purchase completion.

Balance metrics:

- Seconds to first input, bag full, first sale, first upgrade.
- Feathers/min and cash/min by tool/class/party size.
- Round time distribution by chapter/difficulty/party size.
- Upgrade purchase order and level reached.
- Reveal depth and spawn-node completion distribution.
- Failure/leave location.

No economy change should be made from average session time alone. Segment by fresh/returning, party size, class, paid entitlement, platform, and chapter.

19. Security model

- Validate every remote by schema, state, rate, ownership, range, and server time.
- Use monotonic server deadlines for cooldowns.
- Server selects cells and outputs; client never submits yield.
- Server selects RNG result and winner.
- Use integer cents and bounded counts.
- Protect receipts with an idempotent ledger.
- Detect impossible request cadence and impossible movement/action origins; reject first, then sample telemetry.
- Avoid punitive automatic bans from one heuristic. Build evidence and rate-limit.
- Keep Studio-only debug commands disabled outside Studio/allowlisted private test environments.

20. Performance architecture

- Enable Streaming where map structure supports it.

- Pool strands, chunks, particles, number popups, sounds, companion visuals, and temporary UI.
- No per-feather Heartbeat connection.
- Batch pile deltas at configured replication rate.
- Use distance/visibility culling and quality tiers.
- Avoid unanchored decorative physics except short client-only cosmetic bursts.
- Cap concurrent audio and particle systems.
- Disconnect signals and cancel tasks/tweens when controllers/components unload.
- Profile representative worst case: four players, four tools, ambient systems, Egg reveal, victory VFX.

21. Six-phase implementation contract

Phase 1 - Architecture and data

Inspect repository; establish typed config, adapters, networking, profile/session/party/round skeleton, transactions, test framework, and representative ambient/UI foundations. Do not implement the full gameplay loop.

Phase 2 - Complete vertical slice

Deliver lobby -> party -> Chapter 1 -> tutorial -> collect -> sell -> reveal Egg -> victory -> reward -> results -> save -> lobby. Include representative polish and single-player robustness.

Phase 3 - Progression and tools

Implement all Hand/Bag/tools/upgrades/economy/difficulty values and reset/entitlement behavior from config.

Phase 4 - Lobby/meta

Complete parties, classes, rerolls, daily, codes, inventory, stats, chapter selection, responsive modals, and social/group reward.

Phase 5 - Hard, Chapter 2, monetization

Implement config-driven unlocks, complete Chapter 2 puzzles, Marketplace adapters, passes/products, permanent tools, Infinite Bag, event chest, and receipt resilience.

Phase 6 - Production completion

Complete mobile/controller, multiplayer/rejoin, exploit hardening, performance, VFX/audio/environment polish, balance instrumentation, release checklist, and removal of placeholders.

Each phase prompt is standalone and restates the mandatory workflow. Claude Code must implement only the active phase, provide automated and Studio evidence, update docs, commit the verified checkpoint, and stop.

22. Repository and change discipline

- Inspect **AGENTS.md**, README, project files, package manifests, Rojo/Wally config, tests, git status, branches, and existing architecture first.
- Preserve user changes and unrelated dirty files.
- Prefer integration with healthy patterns over creating parallel frameworks.
- No destructive resets or broad deletes.
- Small, reviewable commits; one verified phase checkpoint.

- Update a `docs/IMPLEMENTATION_STATUS.md` in the repository after each phase.
- Record deviations from this pack with reason and affected tests.
- If a material conflict cannot be resolved safely, stop that specific implementation area, report it, and continue only with independent work inside the phase.

23. Definition of done

The game is release-ready only when all six phase gates pass, the full journey works on supported inputs, persistent/premium rewards are idempotent, server authority is demonstrated, performance budgets are met in a representative stress scene, and no dead/unfinished/placeholder space remains in the release path.

Economy reference tables

The complete editable table is supplied as ECONOMY_TABLES.csv. The following compact view highlights core progression and product values. SYSTEM_CONFIG.json remains authoritative.

Bag progression

ID	Level	Value	Cost	Evidence
capacity	1	25 feathers	1.00 cash	VERIFIED
capacity	2	50 feathers	2.00 cash	VERIFIED/ASSUMED
capacity	3	100 feathers	5.00 cash	ASSUMED
capacity	4	200 feathers	12.00 cash	ASSUMED
capacity	5	400 feathers	-	ASSUMED

Tool acquisition

ID	Level	Value	Cost	Evidence
nestRake	0	1 owned_round	8.00 cash	VERIFIED
nestRakeOneRound	0	1 owned_round	19 robux	VERIFIED
nestRakePermanent	0	1 owned_permanent	99 robux	VERIFIED
confettiCharge	0	1 owned_round	25.00 cash	VERIFIED
confettiChargeOneRound	0	1 owned_round	29 robux	ASSUMED
confettiChargePermanent	0	1 owned_permanent	149 robux	CORROBORATED
featherVac	0	1 owned_round	69.99 cash	VERIFIED
featherVacOneRound	0	1 owned_round	49 robux	VERIFIED
featherVacPermanent	0	1 owned_permanent	249 robux	VERIFIED
scoutChick	0	1 owned_round	40 gems	VERIFIED
scoutChickOneRound	0	1 owned_round	19 robux	VERIFIED
scoutChickPermanent	0	1 owned_permanent	129 robux	VERIFIED

Completion rewards

ID	Level	Value	Cost	Evidence
normal	0	1 completion	0	ASSUMED
hard	0	1 completion	0	ASSUMED
normal	0	1 completion	0	ASSUMED
hard	0	1 completion	0	ASSUMED

Daily rewards

ID	Level	Value	Cost	Evidence
day1	1	10 gems	0	VERIFIED
day2	2	120 gems	0	VERIFIED
day3	3	50 gems	0	VERIFIED
day4	4	50 gems	0	VERIFIED
day5	5	50 gems	0	VERIFIED
day6	6	50 gems	0	VERIFIED
day7	7	50 gems	0	VERIFIED
day8	8	500 gems	0	VERIFIED/ASSUMED

Class distribution

ID	Level	Value	Cost	Evidence
newcomer	0	40 percent	40 gems	VERIFIED
nestCarrier	0	25 percent	40 gems	VERIFIED
eggTrader	0	14 percent	40 gems	CORROBORATED
rakeExpert	0	9 percent	40 gems	VERIFIED
blastArtist	0	7 percent	40 gems	CORROBORATED
gemSeeker	0	3.9 percent	40 gems	VERIFIED/ASSUMED
automationExpert	0	1 percent	40 gems	CORROBORATED
masterForager	0	0.1 percent	40 gems	VERIFIED/ASSUMED

Robux products

ID	Level	Value	Cost	Evidence
gemBundle100	0	100 gems	29 robux	VERIFIED
gemBundle400	0	400 gems	69 robux	VERIFIED
doubleGems	0	2 multiplier	79 robux	VERIFIED
doubleLuck	0	2 displayed_luck	199 robux	VERIFIED/ASSUMED
fastRolls	0	1 entitlement	179 robux	VERIFIED
extraClassSlot	0	1 slot	119 robux	VERIFIED
infiniteBag	0	-1 capacity	99 robux	CORROBORATED

Technical implementation appendices

The standalone Markdown files contain the complete working contracts. These summaries preserve the key architecture decisions in the human-readable GDD.

Persistence and transactions

Persistent profiles contain premium currencies, wins/unlocks, best times, classes, permanent perks, cosmetics, daily/code claims, settings, tutorial progress, and aggregate stats. Round cash, bag/tool levels, carried feathers, and one-round grants remain session-scoped. Rewards and developer-product receipts use stable idempotency keys. The server commits reward state before teleport and never starts a disposable writable profile after a failed load.

State-machine invariants

Lobby, party, round, collection, selling, upgrades, Egg reveal, victory, death, rejoin, tools, puzzles, and receipts are explicit state machines. Only valid state transitions accept requests. Victory is atomic, joins close at reveal, and receipt acknowledgement follows persistence.

Verification model

A phase is complete only after automated checks, actual Roblox Studio verification, concrete evidence, updated documentation, and a committed checkpoint. If Studio is inaccessible, Claude Code must say so and stop at the same phase; it may not substitute “should work.”

Search for the Egg - Assumptions Register

This file is the canonical register for decisions that were not directly visible in the supplied 5:25 recording or confirmed by current official evidence. Every item is marked **ASSUMED**. The implementation must read tunable values from `SYSTEM_CONFIG.json`; it must not duplicate these values in services, UI controllers, or tests.

Rules for using assumptions

1. An assumption is a production decision, not a claim about Search For The Needle.
2. Code must refer to a stable configuration key, never a copied literal.
3. Tests may assert the current configured value, but must load the same configuration module generated from the JSON.
4. When later evidence changes a value, update `SYSTEM_CONFIG.json`, `ECONOMY_TABLES.csv`, this register, and the relevant test fixture in one commit.
5. If an assumption changes save semantics, increment the profile schema version and add a migration.

Core round assumptions

ID	ASSUMED decision	Current value	Why this choice is reasonable	Configuration key
A-RND-01	Maximum round lifetime	1,800 seconds	Prevents abandoned servers and pathological sessions while preserving a relaxed count-up timer.	<code>round.maximumRoundSeconds</code>
A-RND-02	Rejoin reservation	120 seconds	Covers common disconnects without keeping stale session membership indefinitely.	<code>round.rejoinReservationSeconds</code>
A-RND-03	Victory flow timing	1.1s lock, 1.5s camera, 2.5s winner, 3s reward	Short, readable, and appropriately celebratory without stalling replay.	<code>round.discoverySequence</code>
A-RND-04	Results auto-continue	15 seconds	Gives players time to read while keeping public parties moving.	<code>round.resultScreenAutoContinueSeconds</code>
A-RND-05	Party win credit	All present party members	Supports cooperation and avoids frustrating last-hit competition.	<code>victory.partyRewardPolicy</code>
A-RND-06	Finder bonus	5 Gems	Adds personal excitement without denying party progression.	<code>victory.finderBonusGems</code>
A-RND-07	Death penalty	No loss; 3-second respawn	Matches a cozy game and avoids losing paid/earned round progress.	<code>death</code>

Egg discovery assumptions

ID	ASSUMED decision	Current value	Rationale	Configuration key
A-EGG-01	Spawn selection	Weighted node, no immediate repeat	Makes QA deterministic enough while reducing obvious repetition.	<code>eggSpawn.selection</code>
A-EGG-02	Candidate node target	24 nodes per chapter	Provides spatial variety without requiring arbitrary voxel positions.	<code>eggSpawn.spawnNodeCountTarget</code>
A-EGG-03	Required pile removal	20%-65% Normal	Produces suspense but avoids a target always near the surface or bottom.	<code>eggSpawn.minimumRemovalFraction</code> , <code>maximumRemovalFraction</code>
A-EGG-04	Reveal and claim radii	5 and 8 studs	Clear local discovery with server-side proximity validation.	<code>eggSpawn.revealRadiusStuds</code> , <code>claimRadiusStuds</code>
A-EGG-05	Claim hold	0.65 seconds	Long enough to prevent accidental claims, short enough to feel responsive.	<code>eggSpawn.claimHoldSeconds</code>
A-EGG-06	One winner event	First valid server claim wins; party shares completion	Prevents duplicate completion and receipt races.	<code>eggSpawn.oneWinnerPerRound</code> , <code>partySharesWin</code>

Economy and progression assumptions

ID	ASSUMED decision	Current value	Rationale	Configuration key
A-ECO-01	Standard feather value	\$0.01	Multiple visible sale notifications support this baseline; the exact rounding rule is assumed.	<code>selling.cashPerStandardFeather</code>
A-ECO-02	Rainbow value	5x	Makes the Lucky Blast proc readable without dominating the economy.	<code>selling.rainbowFeatherMultiplier</code>
A-ECO-03	Bag levels 3-5	100, 200, 400	Clean doubling curve after the verified 25 to 50 step.	<code>bags.levels</code>
A-ECO-04	Bag costs after \$2	\$5, \$12	Creates reachable milestones around tool purchases.	<code>bags.levels[].upgradeCash</code>
A-ECO-05	Remaining Hand curves	See JSON/CSV	Built to produce a meaningful first five-minute upgrade cadence; not claimed as reference values.	<code>handUpgrades</code>
A-ECO-06	Remaining tool curves	See JSON/CSV	Monotonic benefits, increasing costs, no hidden formula.	<code>toolUpgrades</code>
A-ECO-07	Confetti Charge one-round price	29 R\$	Fits the visible ladder between Rake/Drone and Vacuum.	<code>tools.confettiCharge.oneRoundRobux</code>
A-ECO-08	Completion rewards	Chapter/difficulty table	Rewards are deliberately modest because the persistent economy centers on Gems and unlocks.	<code>chapters..Rewards</code>
A-ECO-09	Hard modifiers	1.6x pile, 0.9x yield, 1.1x sale, 2x reward	Makes Hard perceptibly longer and more valuable without requiring separate content.	<code>difficulty.hard</code>
A-ECO-10	Permanent perk effects	+10% per level, capped at 10	First prices are visible; effects/caps were not. Linear effects are transparent and easy to rebalance.	<code>permanentPerks</code>

Class and RNG assumptions

The displayed class weights and roll cost are verified. The renamed target classes preserve function while avoiding copied branding.

ID	ASSUMED decision	Current value	Configuration key
A-CLS-01	2x Luck transformation	Double classes at or below 9% then remove excess probability from Newcomer and normalize	<code>classes.luckTransform</code>
A-CLS-02	Gem Seeker proc	1% gem find; 0.2% diamond feather	<code>classes.weights[gemSeeker].effect</code>
A-CLS-03	Automation swarm proc	5% per completed trip	<code>classes.weights[automationExpert].effect.swarmChancePerTrip</code>
A-CLS-04	Golden Rush proc	3% per minute for 15 seconds	<code>classes.weights[masterForager].effect</code>
A-CLS-05	Duplicate roll behavior	Duplicate is accepted and simply replaces/equips the same class; no pity	Implemented policy; add config if changed
A-CLS-06	Roll result authority	Server-seeded weighted roll; client animation reveals committed result	Architecture rule

Daily, codes, and event assumptions

ID	ASSUMED decision	Current value	Configuration key
A-META-01	Daily claim interval	86,400 seconds	<code>dailyRewardRules.claimCooldownSeconds</code>
A-META-02	Miss grace	48 hours, then reset	<code>dailyRewardRules.gracePeriodSeconds</code>
A-META-03	Day 8 behavior	Loop to Day 1	<code>dailyRewardRules.afterDay8</code>
A-META-04	Group reward	One-time account grant	<code>monetization.groupRewardGems</code> plus saved claim flag
A-META-05	Event skin duplicate conversion	35/50/75/150 tokens by rarity	<code>monetization.eventChest.duplicateTokenValues</code>

ID	ASSUMED decision	Current value	Configuration key	
A-META-06	ALIEN code availability	Enabled until manually disabled	Public current guides support it; expiry was not official.	<code>codes.ALIEN</code>

Chapter 2 assumptions

The hidden key, three colored levers, crystal-count puzzle, four-digit note puzzle, 16-tile rotation puzzle, alien release, and ladder exit are corroborated public structure. Their exact layouts, randomization, timings, and failure behavior are not directly observed.

ID	ASSUMED decision	Current value	Configuration key
A-CH2-01	Target theme	Moonlit Cellar with an original trapped hatchling story	<code>chapters.chapter2</code>
A-CH2-02	Puzzle randomization	New valid solution per round	<code>chapters.chapter2.puzzles.randomizeEachRound</code>
A-CH2-03	Wrong input behavior	Non-destructive feedback; puzzle remains solvable	State machine implementation
A-CH2-04	Chapter 2 Hard unlock	One Chapter 2 Normal win	<code>chapters.chapter2.hardUnlock</code>
A-CH2-05	Party puzzle ownership	Shared world state, any member can interact	Chapter session state

Persistence assumptions

ID	ASSUMED decision	Current value	Rationale
A-DATA-01	Round cash/upgrades	Session-scoped; reset at next round	Matches one-round economy and tool packaging.
A-DATA-02	Gems, tokens, classes, wins, best times, cosmetics	Persistent	These drive the lobby/meta loop.
A-DATA-03	Permanent entitlements	Marketplace ownership is canonical; cached snapshot is diagnostic only	Prevents stale saved ownership from granting products.
A-DATA-04	Rejoin	Restore same active round state within 120 seconds where server/session still exists	Reduces disconnect frustration.
A-DATA-05	Save-before-teleport	Reward transaction is committed before lobby teleport	Prevents win loss and duplicate grants.
A-DATA-06	Failed save	Retry with backoff; show non-blocking warning; queue idempotent reward receipt	Protects progression without trusting client state.

Multiplayer assumptions

ID	ASSUMED decision	Current value	Rationale
A-MP-01	Pile and Egg	Shared by the party	Enables visible cooperation.
A-MP-02	Bag, cash, upgrades	Individual	Preserves personal progression and monetization choices.
A-MP-03	Selling	Individual bag only	Prevents accidental theft of another player's work.
A-MP-04	Tool effects	Remove shared cells; award generated feathers to the acting player	Clear ownership and anti-exploit accounting.
A-MP-05	Completion	All present party members receive base reward; finder receives bonus	Cooperative with a personal moment.
A-MP-06	Late join	Allowed during Searching until the Egg is revealed; no join after Victory begins	Avoids reward sniping.

Mobile and accessibility assumptions

ID	ASSUMED decision	Current value	Configuration key
A-UX-01	Minimum touch target	44 px	<code>polishBudgets.minimumTapTargetPixels</code>
A-UX-02	Mobile visual strand budget	420	<code>collection.pileRepresentation.clientVisualStrandPoolMobile</code>

ID	ASSUMED decision	Current value	Configuration key
A-UX-03	Low-tier target	Stable 30 FPS	polishBudgets.targetFpsMobileLowTier
A-UX-04	Reduced motion	35% camera/UI motion, no essential information removed	polishBudgets.reducedMotionScale
A-UX-05	Controller navigation	Selection groups and explicit focus restoration	UI architecture rule

Premium presentation assumptions

These are deliberate target-game requirements rather than reference-game claims.

- Every interactable uses a four-layer response: anticipation, contact, reward, recovery.
- World activity must be visible while idle: drifting feathers, birds, fans, cloth, machinery, lighting variation, and distant activity.
- Ambient motion is pooled, distance-culled, and disabled or simplified on low quality levels.
- UI uses one consistent visual token set, safe-area layout, responsive scale, haptics where supported, and no overlapping modal ownership.
- Audio is layered by state: lobby, searching, pressure/discovery, victory, and results. Music transitions are cross-faded, not restarted abruptly.
- The vertical slice must contain representative polish. Phase 6 refines and optimizes it; it does not introduce game feel for the first time.

Explicitly disabled until separately designed

The following are not silently assumed into scope: trading, gifting, offline income, subscriptions, rebirth/prestige, global admin events, competitive PvP, and user-generated item exchange. Feature flags remain false. Egg Index and Pets are disabled because the recording advertised an imminent update but did not show the live implementation.

Research sources

Accessed 12 September 2026 unless stated. Public metrics and commerce are volatile. The uploaded recording is primary evidence for all directly observed claims.

No.	Source	Use / limitation
1	User-supplied 5:25.667 screen recording, 1280x720, 60 FPS	Primary current-build observation; exact deployed build ID unavailable
2	Official Roblox experience	Identity, developer, description, root place
3	Rolimon's game record	Universe places, badges, max players; third-party snapshot
4	Roblox Pivot analytics	Dated CCU/visits/likes/favorites snapshot
5	Beebom class guide	Class effect corroboration; weights match live UI
6	GamesRadar codes guide	ALIEN code corroboration
7	TechWiser Basement guide	Chapter 2 route/puzzle corroboration
8	Pro Game Guides Alien Coins	Event currency source hypothesis
9	Pro Game Guides Basement	Independent Chapter 2 corroboration
10	Needle Wiki gamepasses	Historical pass descriptions/prices; live conflicts remain dated

Clean-room restriction

No private source code, script decompilation, private endpoints, exploits, stolen assets, or non-public implementation details are used. Target production must create original code, assets, map construction, writing, branding, models, UI, animation, VFX, and audio.

End of Final GDD